How to get ahead in data science in 3 months

1 message

Danica Simic from Danica's Substack | Data Science, AI & Career <danicasimic@substack.com> 2 October 2026 at 18:48
Reply-To: "Danica Simic from Danica's Substack | Data Science, AI & Career"
<reply+3m35kz&6aijdw&&ba8a34db3904f6d5e651d601015174c347b3fc5d54c709403a26461eb232f1a0@mg1.substack.com>
To: charliecharlie771997@gmail.com

Forwarded this email? Subscribe here for more

How to get ahead in data science in 3 months

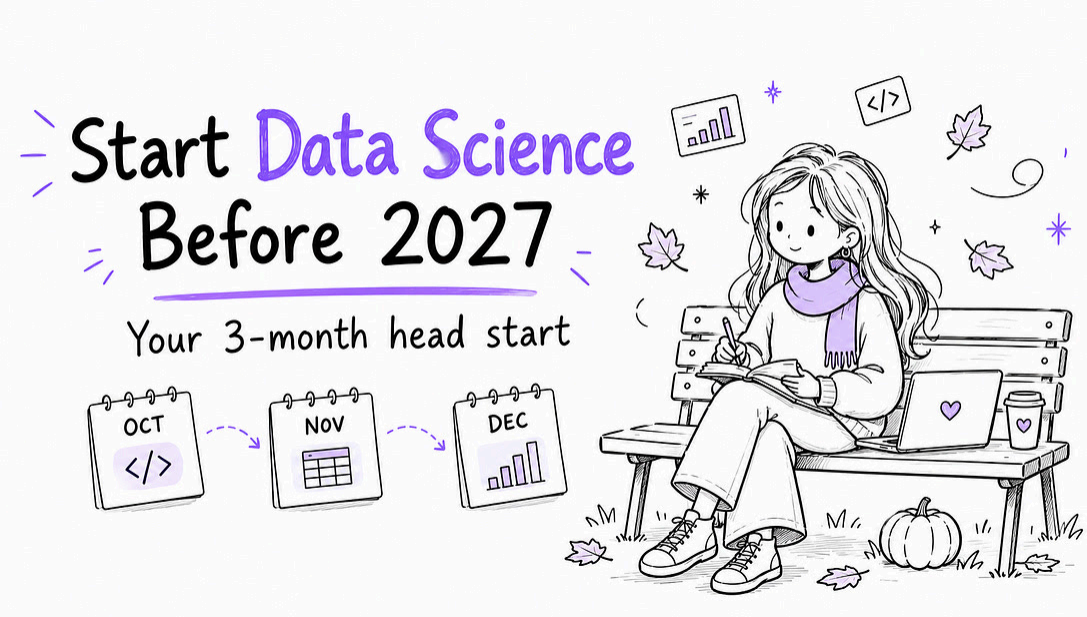
A practical October–January study plan for learning Python, understanding data, and building your first machine learning models.

DANICA SIMIC

OCT 2



READ IN APP



If you start your data science journey on October 1, you won't be job-ready on January 1. However, you'll be 3 months ahead of anyone starting their journey then, so take this advantage on your side.

There are three months left before 2026 ends.

Upgrade to paid

If becoming a data scientist is on your list for 2027, you can use these months to make the beginning much easier.

Imagine opening a dataset on January 1 and already knowing how to inspect it. You can recognize missing values, write a function, build a chart, and explain what a simple model is trying to predict. You still have plenty to learn, but you have somewhere concrete to start.

That is the goal of this guide.

Three months will not give every beginner the same results, and finishing a checklist does not automatically make you ready for a data science job. Your available time, previous experience, and consistency all matter.

But you can build a useful foundation before the year ends. And you can do it without trying to learn every library, collect every certificate, or start five courses at once.

Here is how I would approach it.

Set a pace you can actually maintain

I would plan for around **six to eight focused hours per week**, with extra time if you already enjoy programming and want to go deeper. Treat this as a suggested pace, not a guarantee that every topic will fit neatly into its assigned week.

A simple routine could be four 45-minute weekday sessions, a three-hour weekend practice session, and an hour reviewing what confused you.

Keep roughly half your study time for writing code and solving problems. Watching a lesson helps you recognize an idea; using it without the instructor beside you shows whether you understand it.

Use twelve weeks for the main plan and leave the remaining days in December for catching up and reviewing. If Python takes longer than expected, give it that time. You can continue the machine learning course in January.

Your checkpoints should be things you can do. “I completed six videos” tells you much less than “I wrote a function that summarizes a list of transactions.”

1 Start with Python and use it for small tasks

Suggested focus: weeks 1–2, with practice throughout the plan.

Python is where I would start because it gives you the language you will use to manipulate data and build models later.

At this stage, focus on variables, numbers, strings, booleans, comparisons, conditionals, loops, and functions. Learn how lists, tuples, sets, and dictionaries differ, and practice choosing a structure that fits the task.

For example, a list can hold a sequence of transaction amounts. A dictionary can represent a transaction with a date, category, and amount. A set can help you identify unique categories.

Those distinctions become easier to remember when you use them in something small.

Start with a spending summary. Create a few fictional transactions, total the amounts, and identify the largest purchase. Then add categories and calculate the total for each category. Finally, move repeated logic into a function.

You can also build a temperature converter, a study-hours tracker, or a script that counts words in a text file. If you try file renaming, practice on copies in a dedicated folder first.

The scripts do not need an interface or a clever name. They need to work, and you need to understand them.

Practice readability from the beginning. Use descriptive names such as `total_spending`, keep indentation consistent, and give each function a clear job. Use PEP 8 as a reference for Python style rather than something to memorize before you start coding.

Learn to read errors, too. When a program fails, find the line involved, inspect the values, and describe what you expected to happen. Understanding why a string cannot be added directly to a number is part of learning Python.

Short beginner exercises on HackerRank or LeetCode can supplement your practice. Choose problems involving strings, lists, counting, or simple conditions. Advanced interview puzzles can wait.

Your checkpoint: Write a small script from a blank page that accepts or stores some data, processes it with a loop or function, and returns a useful result. Looking up syntax is fine. Copying the entire solution does not tell you what you can do yet.

Upgrade to paid

2 Learn your notebook environment alongside Python

Suggested focus: start in week 1.

You can learn Python and notebooks together. There is no need to finish one before opening the other.

A notebook lets you combine code, results, charts, and written explanations in one document. That makes it useful for exploring data and recording how you reached a conclusion.

For an easy browser-based start, Google Colab provides a hosted notebook environment without requiring a local installation. Jupyter is another option if

you want to work locally. Choose one for this plan so setup does not become a separate project. [1]

Learn how to create and run code cells, add Markdown cells, restart the runtime or kernel, and run the notebook from top to bottom. Practice loading a small CSV file. If you use Colab, you can also learn to connect Google Drive when you need to keep datasets and results there.

Give your notebook a simple structure: the question you are answering, the data you are using, the steps you took, and what you found.

For your spending exercise, write a short introduction above the code. Under the results, explain which category accounted for the most spending and whether the sample is large enough to support any broader conclusion.

Pay attention to execution order. A cell may work because a variable still exists from something you ran earlier, even if the visible notebook no longer creates it correctly. Before calling your work finished, restart the environment and run everything in order.

Save a copy as an `.ipynb` file. If you later want a static HTML version, Jupyter's export tools can help; you do not need HTML export to complete your first week.

Your checkpoint: Share a notebook that another person can follow, and that runs from a fresh session without relying on hidden variables.

3 Learn NumPy and pandas by asking questions about data

Suggested focus: weeks 3–4.

Once basic Python feels more familiar, start using NumPy and pandas.

NumPy introduces arrays and numerical operations. Learn array creation, indexing, slicing, shapes, reshaping, and calculations such as sums and means. Pay attention to the difference between calculating across rows and across columns. The official beginner guide is a useful reference. [2]

For practice, represent several students' scores across several assignments. Calculate each student's average and each assignment's average. Check the shapes of your results and explain why they differ.

Then move to pandas, where you will spend more time working with tables. Learn the difference between a Series and a DataFrame, how to load a CSV, select columns, filter rows, sort values, and create a new column. The official introductory tutorials provide a focused starting point. [3]

Use one small dataset repeatedly. A fictional shop dataset with dates, products, units sold, and prices is enough to practice useful questions:

- Which product generated the most revenue?
- How many orders were recorded each day?
- Which categories had the highest average order value?
- Are any prices missing or stored as text?
- Are repeated rows duplicates, or could they represent separate purchases?

Get comfortable with `.head()`, `.info()`, `.describe()`, `.value_counts()`, and `.groupby()`. You do not need to recall every argument. You need to recognize which operation could help answer your question.

Treat cleaning as a decision. A missing price is different from a price of zero. A missing customer age may be a reason to investigate the collection process. Removing every incomplete row can change the dataset in ways you did not intend.

For each cleaning step, write down what you changed and why. Inspect the result afterward. How many rows remain? Did a conversion create new missing values? Does the total still make sense?

Your checkpoint: Load a CSV and answer five questions about it. Include a short cleaning log that explains your choices.

Upgrade to paid

4 Learn Seaborn and practice interpreting your charts

Suggested focus: weeks 5–6.

Visualization helps you notice patterns that are difficult to see in a table. It also gives you a way to explain those patterns to someone else.

Start with a few chart types and learn which questions they answer. A histogram shows a distribution. A scatter plot helps you examine a relationship between two numerical variables. A boxplot summarizes a distribution and can make comparisons across groups easier. The Seaborn tutorial covers these families of plots. [4]

Before creating a chart, write its purpose in a sentence. For example: “I want to see whether most orders have similar values or whether a few large orders dominate revenue.” That question gives you a reason to create a histogram.

Then write an interpretation underneath. What does the chart show? What might explain the pattern? What additional information would you need before acting on it?

Separate what you observe from what you assume. Higher sales during a promotion do not, by themselves, prove that the promotion caused the increase. The timing, customer mix, or seasonal demand might also matter.

Try `sns.pairplot()` on a small selection of numerical columns and `sns.heatmap()` for a correlation matrix. Keep them readable. A large grid of every possible relationship rarely helps a beginner explain the important findings.

Learn enough Matplotlib to add titles, axis labels, units, and readable figure sizes. Choose colors that help distinguish groups and remain understandable without relying on color alone.

Use these weeks to learn the statistics behind what you are seeing: mean and median, variation, percentiles, skewed distributions, sample size, and the difference between correlation and causation.

You can connect each concept to the dataset. Compare the mean and median order values, then inspect the histogram. Do a few unusually large orders explain the difference?

Your checkpoint: Produce three useful charts, each with a clear question, a written interpretation, and a limitation. Someone should understand your main findings without reading your code.

5 Begin Andrew Ng's Machine Learning Specialization

Suggested focus: weeks 7–8, continuing as needed.

At this point, you have some experience writing code and investigating a dataset. That gives the machine learning material something to connect to.

Andrew Ng's Machine Learning Specialization, developed by DeepLearning.AI and Stanford Online, is a structured option. Use the newer Python-based specialization for this plan. [5]

One access detail matters: I would not describe the entire experience as unconditionally free. The providers describe audit access, but certificates require paid participation, and access to assignments can depend on the enrollment option. Check the options shown on the official course pages before signing up. A free enrollment button does not mean every feature is included. [5]

For this twelve-week plan, prioritize the regression and classification foundations. Completing the entire specialization can extend into 2027, especially if you are starting programming from scratch.

As you study, keep connecting the terminology to a concrete example. If you are predicting a home's price, the price is the target. Features might include its floor area and location. The model uses patterns in training examples to produce a prediction for another home.

A loss function measures prediction error. Training adjusts model parameters to reduce that error on training data. Gradient descent is one way to make those adjustments.

Learn the intuition, then work through a small numerical example. You do not need to derive everything immediately, but you should be able to explain what is being optimized.

Build the relevant math gradually: averages and variation, vectors and matrices as representations of data, and derivatives as a way to describe change. Revisit the math when an algorithm gives you a reason to use it.

For every major concept, write a short note in your own words and complete a small exercise. Implementing a simple linear prediction and its mean squared error in NumPy is a useful stretch task. Rebuilding every algorithm from scratch is unnecessary at this stage.

Your checkpoint: Explain features, targets, training, validation, and prediction error using one example. Train a simple model and explain how you would check whether it works on unseen data.

6 Understand a few models before adding ensembles

Suggested focus: weeks 9–10.

Spend time with linear regression, logistic regression, and decision trees. Add a basic support vector machine experiment if your schedule allows. You can develop a working understanding now and deepen it through later projects.

Model	First use to study	Question to understand
Linear regression	Predicting a numerical value	How do the inputs contribute to the prediction?
Logistic regression	Predicting a class, often through a probability estimate	How does a threshold turn an estimated probability into a decision?
Decision tree	Classification or regression	How do successive splits divide the examples?
Support vector machine	Start with classification	How does the separating boundary depend on the margin and feature scale?

For each model, learn what it receives, what it predicts, and how it can go wrong. A deep decision tree can fit the training examples extremely closely

and still perform poorly on new ones. Scaling can matter for logistic regression and SVMs, while trees generally do not require it.

Use scikit-learn to practice the complete workflow. Define your target and the information available at prediction time. Set aside test data before making model-development decisions. Fit preprocessing steps on training data, compare models using validation data or cross-validation, and keep the final test set for the end. Pipelines help keep preprocessing and fitting together. [6]

This is where **data leakage** becomes important. If information from the held-out data influences your training process, the evaluation can become too optimistic. For example, filling missing values using an average calculated from the entire dataset allows the held-out rows to influence preprocessing. Learn that average from the training data instead. [6]

Start with a simple baseline. For classification, this could be predicting the most common training class. For regression, it could be predicting the training-set mean. Your model should earn its extra complexity by improving on a relevant baseline.

Choose metrics that match the problem. Classification can involve accuracy, precision, recall, and a confusion matrix. Regression commonly uses MAE or RMSE. A model predicting a numerical wine score and a model predicting whether a wine belongs to a chosen category need different evaluation setups.

Study overfitting, regularization, and a small number of hyperparameters. Try changing a tree's maximum depth and observe what happens to training and validation performance. For logistic regression, explore regularization strength. Change one thing at a time and record what you learned.

Two-feature examples can help you visualize decision boundaries. Then return to the full dataset and remember that a two-dimensional picture cannot capture every aspect of a model with many inputs.

Once you can explain these comparisons, ensemble methods such as random forests and boosting will have a clearer place in your learning.

Your checkpoint: Compare a baseline with two models under the same evaluation setup. Explain the metric, the preprocessing, and at least one reason the apparent winner might fail on different data.

Upgrade to paid

7 Finish a practice project with Titanic or Wine Quality

Suggested focus: weeks 11–12.

Choose one project and finish it. Titanic and Wine Quality can both give you a manageable place to practice the workflow.

These are learning projects. I would not use a standard tutorial-style Titanic or Wine Quality notebook as a flagship portfolio project. Their purpose here is to help you become more independent before developing work that reflects your own questions and interests.

With Titanic, practice binary classification, handling missing values, encoding categorical features, and evaluating a classifier. Establish a local validation process instead of relying only on competition submissions.

With Wine Quality, decide how you will frame the target before training. You can treat the quality score as a numerical prediction task while acknowledging that it is an ordered rating, or define a classification task. If you turn scores into categories, explain the threshold and what information that simplification removes.

Whichever you choose, give your notebook a clear sequence:

1. **Define the question.** What are you predicting, and which inputs would be available when making that prediction?
2. **Describe the data.** Explain what a row represents, what the target means, and the limitations you already know about.
3. **Set up evaluation.** Reserve held-out data before using patterns to guide preprocessing or model choices. Explore the training data in detail.

4. **Prepare the features.** Handle missing values, encode categories, and scale when appropriate. Fit learned transformations only on training data.
5. **Build a baseline and compare models.** Keep the data splits and metrics consistent so the comparison is meaningful.
6. **Inspect errors.** Look for examples or groups where the model struggles. Consider whether small sample sizes make those observations uncertain.
7. **Write your findings.** Explain the result, limitations, and the next experiment you would try.

Your report can be short. Aim to answer: What did I try? What happened? How reliable is that result? What would I investigate next?

Avoid polishing the notebook before the analysis works. Finish the basic version, restart the environment, run every cell, and check that your written conclusions match the outputs.

Your checkpoint: Complete one notebook you can walk through without a tutorial open. You should be able to explain your decisions and identify what you still do not understand.

Your twelve week plan at a glance

Week	Main focus	Something to finish
1	Python basics and notebooks	A notebook with small calculations and explanations
2	Functions and data structures	A working spending or study tracker
3	NumPy and introductory pandas	Array exercises and a loaded CSV
4	Cleaning and summarizing tables	Five answered questions and a cleaning log
5	Seaborn and chart selection	Three charts tied to specific questions
6	EDA and descriptive statistics	A short analysis with findings and limitations
7	Regression foundations	Notes and a simple regression experiment
8	Classification and evaluation	A classifier compared with a baseline
9	Logistic regression and trees	A consistent comparison of two models
10	Validation and tuning, with SVMs if time allows	An experiment log explaining what changed
11	Titanic or Wine Quality	A complete first project draft
12	Error analysis and communication	A reproducible notebook and short report

Use the remaining December days to revisit gaps. Repeating a difficult exercise is useful progress.

Use AI to support the learning process

If you use an AI assistant, give it a role that keeps you actively involved.

Ask it to explain an error, question your reasoning, or give you a similar exercise. For example: “Give me one hint about why this function returns the wrong total. Let me try the fix before you show the solution.”

You can also ask it to quiz you on your notebook: “Ask me why I chose this metric and how my preprocessing could leak information.”

Check explanations against documentation and your own experiments. AI-generated code can run successfully while solving the wrong problem.

After receiving help, close the answer and reproduce the step yourself. If you cannot explain a line, keep investigating it. That is how the assistance becomes part of your learning.

> This doesn't mean that you should become dependent on AI. Let it give you exercises, test you and ask questions about your project that you'll be able to explain in your own words.

Upgrade to paid

What you should carry into January

By the end of this plan, you are aiming for a set of practical abilities: writing small Python programs, exploring tables, making useful charts, training basic models, and explaining an evaluation honestly.

Keep your scripts, your EDA notebook, your completed practice project, and a list of questions that remain unclear. Those give you evidence of progress and a starting point for the next stage.

SQL deserves dedicated attention next, alongside deeper statistics and probability. Then develop an original project around a question you care about, with data choices, evaluation, and limitations you can defend. Basic Git skills will help you organize and share that work.

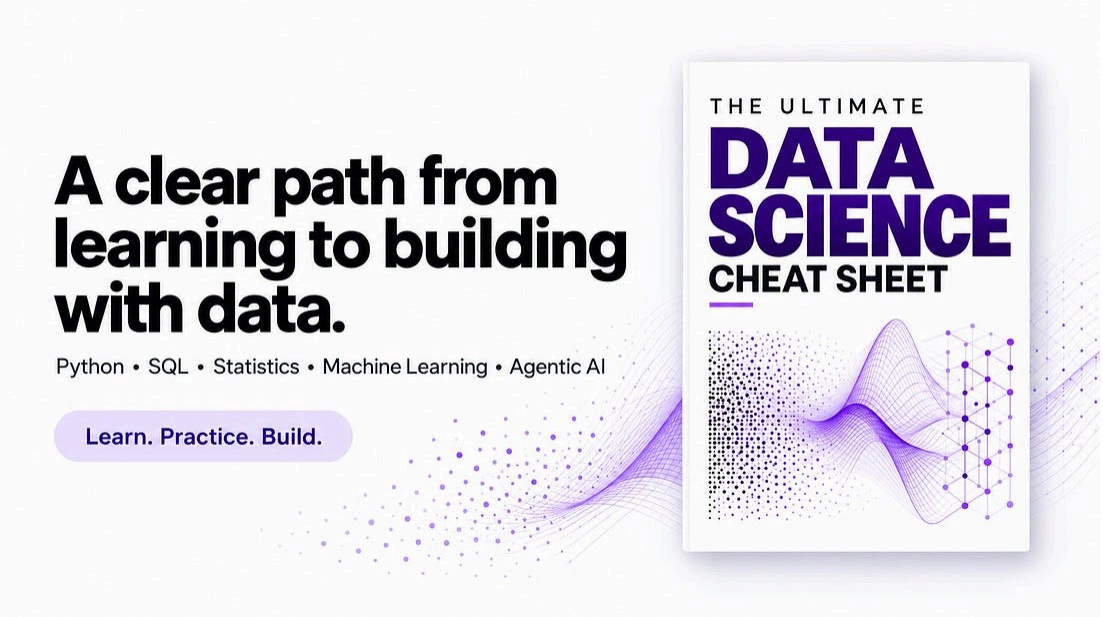
You do not need to have the entire career mapped out before beginning. You need a study session you can keep and a task you can finish.

Today, open a notebook, create a small list of numbers, and write a function that tells you something useful about them. Add a sentence explaining what you did. Then schedule the next session.

January 1 can arrive with twelve weeks of practice already behind you.

Where are you starting: Python, data analysis, or your first machine learning model? Reply and tell me which part you want to feel more confident about by the end of December.

Still don't know where to start and what to focus on? Get lifetime access to my Data Science Cheat Sheet to help guide you and build with you.



Data Science Cheat Sheet

Resources mentioned in this guide

1. Google Colab — hosted notebooks and getting started information.
2. NumPy beginner guide — arrays, shapes, indexing, and numerical operations.
3. pandas introductory tutorials — selecting, summarizing, and manipulating tabular data.
4. Seaborn tutorial — distributions, relationships, and chart selection.
5. Machine Learning Specialization on Coursera and DeepLearning.AI course information — course details and enrollment options. Access terms should be checked when enrolling.
6. scikit-learn common pitfalls — preprocessing, leakage, and consistent evaluation.

You're currently a free subscriber to Danica's Substack | Data Science, AI & Career. For the full experience, upgrade your subscription.

Upgrade to paid

LIKE COMMENT RESTACK

© 2026 Danica Simic
548 Market Street PMB 72296, San Francisco, CA 94104
Unsubscribe

Get the app Start writing